

Explaining Rankings with Hidden Group Bonuses

Alvin Hong Yao Yan
National University of Singapore
Singapore, Singapore
alviny@u.nus.edu

Suraj Shetiya
Indian Institute of Technology Bombay
Mumbai, India
surajs@cse.iitb.ac.in

Sujoy Bhore
Indian Institute of Technology Bombay
Mumbai, India
sujoy@cse.iitb.ac.in

Priyanka Golia
Indian Institute of Technology Delhi
Delhi, India
pgolia@cse.iitd.ac.in

Diptarka Chakraborty
National University of Singapore
Singapore, Singapore
diptarka@nus.edu.sg

Abstract

Determining a linear utility function that correlates with observed candidate rankings is a foundational problem with applications in domains such as admissions, hiring, and recommendation systems, e.g., [Storandt and Funke, AAAI'19, Zhang et al., KDD'23, Wang et al., ICDE'24 (best paper award), Chen and Wong, VLDB'24]. Traditionally, these models assume full visibility into the feature sets used to determine the utility score. However, real-world scenarios often involve sensitive attributes that are hidden or partially observed, yet may influence outcomes through additive bonuses designed to promote fairness, as in [Gale and Marian, ICDE'24]. Motivated by such practical concerns, we study a variant of the ranking explanation problem where sensitive features are unobserved but may influence candidate rankings through group-specific linear boosts.

We present a formal framework for modeling this problem and develop an algorithmic solution that leverages constraint satisfaction and automated reasoning techniques to jointly infer the linear scoring parameters and latent group bonuses consistent with the observed rankings. We further show that determining a satisfying linear function with group-specific bonuses is NP-hard in general, but when the feature dimension and the number of groups are constant, the problem admits a polynomial-time solution. Our approach is the first to address this nuanced variant, which captures key real-world challenges in fair ranking and admission systems. We perform extensive experiments on both real-world and synthetic datasets, demonstrating that our method effectively recovers hidden bonus structures and provides faithful explanations of observed ranking outcomes.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; *Learning to rank*; • **Social and professional topics** → User characteristics.

Keywords

Explainable Ranking, Linear Utility Models, Hidden Group Bonuses, Hyperplane Arrangements, Mixed Integer Linear Programming

ACM Reference Format:

Alvin Hong Yao Yan, Suraj Shetiya, Sujoy Bhore, Priyanka Golia, and Diptarka Chakraborty. 2026. Explaining Rankings with Hidden Group Bonuses. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3770855.3818145>

Resource Availability:

The source code associated with this paper is publicly available at <https://doi.org/10.5281/zenodo.20322450>.

1 Introduction

Ranking algorithms are central to numerous AI applications, including web search, recommender systems, and information retrieval. However, despite their pervasive use, the decision-making processes behind these algorithms often remain opaque, which raises critical concerns around trust, fairness, and accountability. As a result, explainability in ranking has emerged as a pressing research challenge within the AI community. Several recent works have sought to adapt principles from interpretable machine learning to the ranking setting [12, 13, 26], aiming to generate faithful, user-understandable explanations for ranked outputs. Some approaches focus on post-hoc explanations, while others design models that are easier to interpret from the start [1, 50]. There is also increasing attention to broader goals like fairness and trust in ranked results [16, 44]. Yet, many basic questions remain open: *what does it mean for a ranking to be explainable? How do we measure the quality of an explanation when the output is an ordered list instead of a single label?*

Modeling user preferences is a central challenge in multi-criteria decision-making. A common and effective approach is to formalize these preferences with a *utility function* that assigns a numerical score/utility to each candidate, thereby inducing a natural ordering. Candidates are typically characterized by a set of features, which can be distinguished as *scoring attributes* – that directly contribute to utility, and *sensitive attributes* – used to enforce auxiliary goals such as fairness or diversity. Among various functional forms, linear utility models are arguably the most widely adopted. Their prevalence stems from a powerful combination of simplicity and practical efficacy. The straightforward structure of a weighted sum not only facilitates deployment in large-scale systems like web search and recommender engines [24, 37, 53] but also offers inherent transparency [21, 47]. This transparency, simplicity, and effectiveness make linear utility-based models a natural starting point for research



This work is licensed under a Creative Commons Attribution 4.0 International License. KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3818145>

Candidate	Group	Test Score	SAT Score	Rank
c_1	-	9.8	2.0	3
c_2	-	8.1	7.8	1
c_3	-	7.2	8.6	2
c_4	-	6.9	4.2	7
c_5	UP	6.0	3.2	4
c_6	UP	3.7	7.1	5
c_7	-	5.1	8.0	6
c_8	UP	4.5	3.5	8

Table 1: Example dataset of eight university applicants.

into explainability and learning to rank [4, 10, 11], interactive learning [38, 59, 61], reverse regret [57] (best paper award at ICDE'24), and the integration of fairness and diversity mechanisms [23, 31, 58]. Despite their simplicity, linear utilities have demonstrated competitive performance, confirming their status as a robust and versatile modeling choice, e.g., [42, 52]. We refer the readers to Section 7 for a more elaborate discussion on related works.

Several works have studied how to quantify the distance between a given ranking and one induced by a (linear) utility function. Common notions include the number of violated pairwise orders, the distance to a consistent utility vector, and the minimum perturbation needed to make the ranking “linearizable”. Exact computation is often intractable in the presence of conflicting preferences, motivating the study of approximation and structural relaxations (e.g., [15, 17]).

In many real-life scenarios, a small additive bonus is essential to ensure that a linear utility-based ranking model aligns with observed outcomes. Such a bonus is primarily calculated based on the sensitive attributes [3, 49, 58] – separate from the scoring attributes used to compute utility – associated with each candidate or item. This often arises when external goals, such as affirmative action, fairness constraints, or prior trust scores, must be incorporated uniformly across items of certain groups. The application of compensatory additive bonuses to linear utilities is an approach that has been explored for promoting fairness in ranking systems [22]. Without such a term, even simple adjustments to promote underrepresented groups may not be achievable within the linear feature space, making the additive bonus a minimal yet powerful mechanism for enforcing policy-driven interventions (see [34]).

We motivate our problem with the following running example of diversity in University admissions.

Example 1 (University admissions). Consider a public university conducting its annual admissions, where applicants are ranked using a *linear utility-based model*. Academic performance is assessed through multiple indicators, such as entrance test and SAT scores, which are normalized, weighted, and linearly aggregated to compute a base utility score. As a public institution committed to diversity, equity, and inclusion (DEI), the university augments this model with an *additive incentive mechanism*. Recognizing that applicants from underprivileged (UP) backgrounds may face structural disadvantages affecting academic performance, an expert committee assigns a group-specific bonus score. This bonus is added to the base utility score of each applicant in the corresponding group, yielding an adjusted utility score that determines the final ranking.

For example, suppose eight applicants apply, as shown in Table 1. The candidates are ranked $c_2, c_3, c_1, c_5, c_6, c_7, c_4, c_8$ based on their adjusted utility scores. Given only anonymized public data (academic indicators and rankings), third-party auditors aim to explain the observed outcomes under this *linear utility function with additive group-level bonuses*. The sensitive attributes, such as group membership (Column 2 of Table 1), and the exact bonus values are not disclosed.

Problem (Informal) Formulation. Given a ranking, can it be explained by a linear utility function with a small number of additive bonuses? In the SINGLETON GROUP BONUS LINEAR RANKING (SGBLR) version, arbitrary bonuses may be given to at most k items. In the g -GROUP BONUS LINEAR RANKING (g -GBLR) version, there are g (special) groups each containing a limited number of items, and group-specific bonuses – each item of a particular group gets the same amount as a bonus, and bonus value for different groups could be different – are allowed. The goal is to find weights and bonus assignments that reproduce the ranking exactly.

Note that Example 1 is for the 1-GBLR variant with one (special) group being UP – all the candidates of the UP group receive an additive bonus of the same amount.

Our method conceptually builds upon seminal works LIME [45] and SHAP [33], which infer feature contributions to predictions, as our model learns weights for each feature, which provides an explanation for the observed ranking. It extends this line of work by modeling hidden additive bonuses beyond observable features as part of the explanation. We envision our technique as contributing to the broader area of trustworthy and responsible data science, particularly in applications such as algorithmic auditing and explainability. For example, in an auditing setting, suppose a ranking model is used to evaluate candidates, and there is a requirement that all candidates within the same group receive identical bonuses. If our method is unable to recover a linear utility function with such additive bonuses that explains the observed ranking, this serves as evidence that the ranking violates the intended fairness constraint. Alternatively, our framework can be used for interpretability. If an observed ranking can be well-approximated by our model, the learned weights provide insight into the relative importance of different attributes, while the inferred additive bonuses reveal how much advantage may have been assigned to specific candidates. This can help assess whether such adjustments are reasonable within the given domain and context.

1.1 Our Contributions

In this paper, we initiate a formal study on the problem of learning a linear utility function with small additive group-wise bonuses that realizes a given ranking. On the theoretical side, we provide exact algorithms and establish computational hardness. Next, we address the question of designing scalable algorithms that work efficiently in practice. More specifically, we show the following results.

- We start by considering the singleton group variant (SGBLR) that captures the allowance of arbitrary (potentially different) additive bonuses to a few individual items. We design a hyperplane-based algorithm – enumerating over all the regions specified by a set of comparison hyperplanes, and then performing certain *longest common subsequence* (LCS) computation in each region. Our algorithm works in polynomial time for constant-dimensional points. We

further extend our algorithm to a more general variant of group-wise bonuses (g -GBLR) by introducing a new auxiliary dimension per group, leading to an algorithm that runs in polynomial time for a constant number of groups and for constant-dimensional points. Note that often in practice, both the number of groups and the dimension of the feature vectors are indeed small (bounded by some constant).

- We complement our algorithmic results with computational hardness. The problem, even if in its simplest form of singleton groups, is NP-hard for arbitrary dimensions, establishing that the exponential dependency on dimension of our algorithm is unavoidable assuming $P \neq NP$.
- While the theoretical algorithm establishes tractability in principle at least for constant dimension, its high complexity motivates us to explore more practical alternatives using modern ILP solvers. We formulate the problem as an MILP and evaluate its performance using state-of-the-art solvers.
- We conduct extensive experiments on both real-world and synthetic datasets to demonstrate the efficacy and scalability of our methods. In case of SGBLR we observe that for two-dimensional data the MILP formulations scale well, whereas the computational hardness inherent in the hyperplane-based algorithm prevents it from scaling beyond small instances. In the case of g -GBLR with $g=1$, the MILP formulations exhibit strong scalability with respect to both dimensionality (up to $d=17$) and the number of tuples (up to approximately 75,000). For the real-world dataset, containing around 300,000 tuples, the MILP formulations successfully handle the g -GBLR problem for both $g=1$ and $g=2$, highlighting their practical applicability at large scale.

2 Preliminaries

In this section, we briefly introduce the notations used in the paper and then formally describe the problem formulation.

Data model Consider a dataset $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ of n tuples, each with d numerical attributes $\mathcal{A} = A_1, A_2, \dots, A_d$ and additional categorical attributes (such as *gender* and *race*), some of which are sensitive and not revealed during the ranking process. We use $t_i[j]$ to denote the value of tuple t_i for attribute A_j .

Utility model As part of the black-box scoring process, each tuple $t_i \in \mathcal{D}$ is scored based on its values for the numerical attributes $\mathcal{A}$. While there are numerous “models” that have been used to score tuples (such as *monotone* [20], *linear* [8, 14]), we adapt the widely used linear utility functions to this work, as in several previous works in the literature, e.g., [10, 57, 58]. *Linear utility function* s_w consists of a d dimensional weight vector $w = \{w_1, w_2, \dots, w_d\}$. Given a linear function s_w , the score/utility for tuple t is computed as $s_w(t) = \sum_{j=1}^d w_j \cdot t[j]$. As each attribute in the dataset has a non-zero contribution towards scoring, $\forall j \in [d] \ w_j \neq 0$.

Without loss of generality, we assume that $\forall j \in [d] \ w_j \geq 0$ and $\sum_{j \in [d]} w_j = 1$; such assumptions are justified in [38, 59]. For the sake of completeness, we provide a brief discussion on positive weights in Appendix B.

Bonus-aware utility function: Based on certain attributes which are termed as *sensitive attributes*, bonus scores may be allotted to items which belong to certain groups.

In our running example, under-privileged candidates in the dataset would receive an additive bonus based on historical biases. We model this as an additive bonus on top of the linear utility function score. All tuples sharing the same sensitive attribute τ receive the same bonus, denoted by $\text{bonus}(t[\tau])$.

Based on this formulation, the bonus-aware utility function, which accounts for the sensitive attributes, is given by,

$$f_w(t) = \text{bonus}(t[\tau]) + \sum_{j=1}^d w_j \cdot t[j] \quad (1)$$

For simplicity, we denote the bonus-based utility function f_w as f throughout the paper.

To illustrate, in Example 1, the bonus-based utility function for the tuples in the under-privileged group UP is given by $f_w(t) = 2 \times \text{test} + 1 \times \text{SAT} + 5$, where 5 is the additive bonus given only to UP candidates.

Ranking: The scores of the tuples using the bonus-based linear utility function are used to rank the tuples. As a consequence of the ranking process, a complete ranking over the n input tuples can be observed. We refer to this ranking as π .

Realizable by linear functions: Given a dataset $\mathcal{D}$ and a ranking π , we say that π is *realizable* if there exists a linear utility function s_w such that applying s_w to $\mathcal{D}$ produces the ranking π . Importantly, this function s_w does not involve any additive bonuses. We denote such a function as $s^{-1}(\pi)$.

If no such realizable linear function exists, it indicates that the observed ranking π must have been generated using the black-box ranking process that includes some form of additive bonus. In this case, we aim to explain π by constructing a linear function augmented with additive group-based bonuses. As part of this process, the dataset is partitioned into “groups” and a fixed bonus is assigned to each group, such that a linear function with these bonuses can realize π .

Without any constraint on the number of groups, one can trivially assign each tuple to its own “under-represented” group and learn arbitrary bonuses. To avoid such degenerate solutions, we restrict the number of groups to at most g .

Realizable by bonus-aware linear functions: Given a dataset $\mathcal{D}$, a ranking π , and a budget of $g+1$ groups, we say that π is *realizable by a bonus-aware linear function* if there exists a partition of the tuples into $g+1$ groups, where the majority group receives a bonus of 0, such that a linear function f_w , combined with these group bonuses, produces the ranking π . We denote such a function as $f_g^{-1}(\pi)$.

We are now ready to formally introduce the two problems that form the focus of this work.

Problem formulation. For our first problem setting, we consider finding a bonus-aware linear function that realizes a given ranking π where the number of tuples that receive a bonus is at most k .

Definition 2 (SGBLR). Given a set of tuples $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ where each $t_i \in \mathbb{R}^d$, a target ranking π over the tuples and an integer k , does there exist a linear utility function s_w with $w \in \mathbb{R}^d$ and a bonus vector $\mathbf{v} \in \mathbb{R}^n$ with at most k non-zero values such that sorting the tuples by $s_w(t_i) + v_i$ produces the ranking π ?

In practice, an estimate of the number of tuples belonging to under-represented groups is often available. The SGBLR problem captures the lower bound on the number of tuples that need to be allotted a bonus for a linear function to realize π .

The number of under-represented groups is typically a small known number (usually constant), and thus it would be of interest to

find a bonus-based linear utility function that respects the number of groups g . We now propose the g -GBLR problem.

Definition 3 (g -GBLR). Given a set of tuples $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ where each $t_i \in \mathbb{R}^d$, a ranking π over $\mathcal{D}$ and integer k . Does there exist a weight vector $w \in \mathbb{R}^d$, bonus values $v_1, \dots, v_g \in \mathbb{R}^+$, and disjoint subsets $G_1, \dots, G_g \subseteq \mathcal{D}$ where $\sum_{i \in [g]} |G_i| \leq k$, such that ranking π can be realized by s_w using v_i as bonus values for tuples in G_i ?

Note, some tuples in $\mathcal{D}$ may not belong to $\cup_{i \in [g]} G_i$.

3 Algorithmic Framework

3.1 Algorithmic Results for SGBLR

We start by considering the SGBLR problem and provide an algorithm to solve this problem.

THEOREM 4. *There exists a deterministic algorithm that, given a set of tuples $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ where each $t_i \in \mathbb{R}^d$, a ranking π over $\mathcal{D}$ and an integer k , if there exists a satisfying solution to SGBLR, returns a linear utility function s_w with $w \in \mathbb{R}^d$ and a bonus vector $v \in \mathbb{R}^n$ with at most k non-zero values such that sorting the tuples by $s_w(t_i) + v_i$ produces the ranking π , in time $O(n^{2d+1} \log n)$.*

We devote the rest of the subsection to proving the above theorem. Before describing our approach, we introduce the notion of “comparison hyperplane” and then propose algorithms which explore regions of the weight-space based on comparison hyperplane.

Definition 5 (Comparison hyperplane). Consider two tuples $t_i \in \mathbb{R}^d$ and $t_j \in \mathbb{R}^d$. The set of all weight vectors $C_{i,j}$ where the tuple t_i has a score equal to t_j is defined as a comparison hyperplane. Formally, $\sum_{a=1}^d w_a \cdot t_i[a] = \sum_{a=1}^d w_a \cdot t_j[a]$.

Given two tuples t_i and t_j , a comparison hyperplane divides the d -dimensional weight space into two parts such that for any weight vector in $C_{i,j}^+$, t_i is better than t_j , i.e.,

$$C_{i,j}^+ = \{w \in \mathbb{R}^d \mid \sum_{a=1}^d w_a \cdot t_i[a] > \sum_{a=1}^d w_a \cdot t_j[a]\}, \quad (2)$$

and for any weight vector on the other side $C_{i,j}^-$, t_j is ranked better than t_i .

Our idea stems from a simple observation that, while there are $n!$ orderings of n tuples, in d -dimensional space, not all of these are realizable through linear utility functions. Instead, we will soon see that only n^{2d-2} rankings are realizable. We then exploit this property to design an efficient algorithm for both problems.

Algorithm for SGBLR. We first present our approach, and then provide an argument about its correctness. Consider the d -dimensional weight-space. For every pair of tuples t_i, t_j introduce the comparison hyperplane $C_{i,j}$ in this space. We term the set of these $\binom{n}{2}$ hyperplanes as $\mathcal{H}$. Introduction of $\mathcal{H}$ to the $\mathbb{R}^d$ dimensional weight-space partitions it into numerous regions, which is termed the arrangement of hyperplanes [18]. As each weight-function in this space lies either below or above the comparison hyperplanes, the ranking that is realized corresponds to a unique ranking.

The set of all functions that have the same ranking in the arrangement can be shown to be a convex region and corresponds to a “cell” of the arrangement. To formalize this, consider a vector of $\binom{n}{2}$ boolean signs corresponding to each utility function termed as the

Algorithm 1 ERMB : ENUMERATE RANKINGS AND MINIMIZE BONUSES

```

1: procedure ERMB(Dataset  $\mathcal{D}$ , target ranking  $\sigma$ )
2:   Initialize  $\mathcal{H} \leftarrow \emptyset$ 
3:   for all pairs  $(t_i, t_j)$  where  $1 \leq i < j \leq n$  do
4:     Add comparison hyperplane  $C_{i,j}$  to  $\mathcal{H}$ 
5:   Enumerate regions formed by the arrangement of  $\mathcal{H}$ 
6:   Let  $\mathcal{R} \leftarrow$  all rankings  $\rho$  realizable in each region of  $\mathcal{H}$ 
7:   Initialize  $\rho^* \leftarrow \text{null}$ ,  $\text{maxLCS} \leftarrow 0$ ,  $\mathcal{G} \leftarrow \text{null}$ 
8:   for all rankings  $\rho \in \mathcal{R}$  do
9:     Compute  $L \leftarrow \text{LCS}(\rho, \pi)$ 
10:    if  $|L| > \text{maxLCS}$  then
11:       $\text{maxLCS} \leftarrow |L|$ ;
12:       $\rho^* \leftarrow \rho$ ;
13:       $\mathcal{G} \leftarrow [n] - L$ ;
14:   return linear function  $s^{-1}(\rho^*), \mathcal{G}$ 
```

location vector. A location vector is of the form $\{+, -\}^{\binom{n}{2}}$. For example, the (i, j) -th location contains the sign $-$ or $+$ which represents the location of the weight vector with respect to $C_{i,j}$, i.e., if the sign is $+$, then the tuple t_i is scored better than t_j . A similar argument follows for $-$. The set of ranking functions with the same location vector can be found by the intersection of $\binom{n}{2}$ half-spaces; the set of ranking functions is convex. Also, note that this corresponds by definition to the cell of an arrangement [18].

Based on this, our approach to solve SGBLR is to enumerate the regions in space formed by $\mathcal{H}$, and in turn, enumerate the realizable rankings, $\mathcal{R}$. For every ranking $\rho \in \mathcal{R}$, we compute the least number of tuples that need to be allotted a bonus to obtain the ranking π , which is equivalent to finding a *Longest Common Subsequence (LCS)* between π and ρ . Since both the inputs to the LCS problems are rankings, it is well-known that the problem is essentially the same as finding a *Longest Increasing Subsequence (LIS)* in ρ where the reference ranking is π – by renaming the tuples in ascending order based on π , i.e., we give the tuple located at $\pi[0]$ a new number of t_0 and so forth.

The tuples that are not part of the LIS need to be allotted a bonus so that the linear ranking function $s^{-1}(\rho)$ can realize π . Over the set of rankings $\rho \in \mathcal{R}$, our algorithm outputs the linear function that had the longest value for the increasing subsequence. The tuples that are not part of the subsequence need to be added with a bonus to obtain π . We present the pseudo-code for our approach in ERMB (Algorithm 1).

Analysis of ERMB. The analysis of our algorithm relies on two steps: (i) enumeration of the cells of the arrangement, and (ii) LIS of the sequences ρ . To prove the complexity of the enumeration step, we rely on Corollary 28.1.2 of [54]. The corollary states that the number of such regions with n hyperplanes in d -dimensional space is bounded by $O(n^d)$. We state the Corollary below for easy access. **Proposition 6.** [54] *The maximum combinatorial complexity of an arrangement of n hyperplanes in $\mathbb{R}^d$ is $O(n^d)$. Moreover, if the arrangement is simple, its complexity is indeed $\Theta(n^d)$.*

Observe that each hyperplane $C_{i,j}$ as shown in Equation 2 has an intercept of 0 and hence passes through the origin. Therefore, the resulting arrangement is a special case of a hyperplane arrangement known as a central arrangement (see [46] for details). In a central arrangement, the combinatorial complexity of a central arrangement with m hyperplanes in d dimensions is $O(m^{d-1})$ (see [46] Chapter 5).

Thus, in our case, n^2 hyperplanes in d -dimensional weight space form a central arrangement, which bounds the number of unique rankings realizable to $O(n^{2(d-1)})$. This helps us in establishing $O(n^{2d+1}\log n)$ bound on the time complexity (see Appendix C).

Next, we show the correctness of our algorithm ERMB (Algorithm 1), which will complete the proof of Theorem 4.

LEMMA 7. ERMB (Algorithm 1) returns a linear utility function with weight vector $w \in \mathbb{R}^d$ which solves the SGBLR problem.

PROOF. Suppose vectors w^*, v^* represent the optimal weight vector and optimal bonus values, respectively. The linear function f_{w^*} with bonus of v^* orders $t_i \in \mathcal{D}$ by decreasing value such that the ordering matches π and v^* is the most sparse among all such solutions.

Now, consider the region of the arrangement of $C_{i,j}$ where the vector w^* lies in. Let w be the vector in this region that our algorithm picks, and let ρ be the resulting ranking from ordering $t \in \mathcal{D}$ in decreasing order of their scores. For any pair of points $t_1, t_2 \in \mathcal{D}$ that receive no bonus from v^* , it is clear that $w \cdot t_1 > w \cdot t_2$ if and only if $w^* \cdot p_1 > w^* \cdot p_2$. Therefore, both ρ and π must have the same total order over points in $\mathcal{D}$ that receive no bonus from v^* . These points are a lower bound on the length of the LCS of π and ρ , so our approach must find a w, v which satisfies SGBLR with v at least as sparse as v^* . $\square$

Improved Running Time for 2-dimensions. We remark that in the special case of $d = 2$, the enumeration step (line 4 of Algorithm 1) can be performed very efficiently as the weight space is now 1-dimensional. Therefore, each comparison hyperplane is a point partitioning the line. Enumeration of the regions can thus be done in $O(n^2 \log n)$ time by sorting the comparison hyperplanes, followed by a linear scan to obtain the ranking and then performing a LIS on each ranking, leading to an overall running time of $O(n^3 \log n)$ (compared to an worse bound of $O(n^5 \log n)$ implied from Theorem 4). We use this faster implementation in our experiments.

3.2 Algorithm for g -GBLR

The budgeted group setting is more intriguing as Algorithm 1 cannot directly be adapted to this setting, because the notion of a fixed number of groups is not modeled in SGBLR. In this section, we first embed the idea of *bonus* into the weight space and then design a polynomial-time algorithm for the fixed dimension, fixed group setting.

We treat group bonuses as *bonus dimensions*, expanding the feature space to $d+g$ dimensions – one additional dimension per group. Then, to handle group membership, the algorithm generates $g+1$ copies of each tuple: one base version and one for each group membership (whether it belongs to a particular group or not). It then uses hyperplane arrangements in this augmented space to enumerate all possible rankings of the expanded dataset. Unlike simpler cases that use Longest Increasing Subsequence, this approach requires us to employ a budgeted Longest Common Subsequence (LCS). A dynamic programming routine aligns the target ranking π with these candidate rankings to find the optimal solution that strictly respects the group size constraints. We provide a detailed description of the algorithm and analysis in Appendix D.

4 Hardness Results

In this section, we show that the problem of linear realizability of a ranking with additive group bonuses is NP-hard.

THEOREM 8. SGBLR is NP-hard.

To show the above, we provide a reduction from a variant of the classical MAX-2-SAT, namely MAX-1-IN-2-SAT.

Definition 9 (MAX-1-IN-2-SAT). Given a 2-CNF formula, and a non-negative integer r , the problem is to decide whether there exists an assignment of variables that satisfies at least r clauses by making *exactly one* literal (out of two literals) in each of these r clauses to be true.

The above problem is known to be NP-hard by a straightforward reduction from MAXCUT – another classical NP-hard problem. In fact, the problem remains NP-hard even for monotone formulas.

THEOREM 10 (FOLKLORE). MAX-1-IN-2-SAT is NP-hard.

Reduction from MAX-1-IN-2-SAT. Suppose we are given an instance of the MAX-1-IN-2-SAT problem: A 2-CNF formula F over n variables $x_1, \dots, x_n$, with m clauses $C_1, C_2, \dots, C_m$ (without loss of generality assume that $m < n^2$), and a non-negative integer r . Then we construct an instance of the decision version of the SGBLR problem as follows: For each clause C_i , create a point $p_i \in \mathbb{R}^n$ by setting,

$$p_i[j] = \begin{cases} 1 & \text{if } x_j \text{ appears in } C_i \\ -1 & \text{if } \bar{x}_j \text{ appears in } C_i \\ 0 & \text{otherwise.} \end{cases}$$

Further, take $\ell = (m+1)n^2$ additional points: $q_1 = \dots = q_\ell = 0^n$. Consider the dataset $\mathcal{D} := \cup_{i=1}^m \{p_i\} \cup \cup_{i=1}^{\ell} \{q_i\}$. Then, consider the following ranking $\pi := q_1 \preceq \dots \preceq q_{n^2} \preceq p_1 \preceq q_{n^2+1} \preceq \dots \preceq q_{2n^2} \preceq p_2 \preceq \dots \preceq q_{\ell-2n^2+1} \preceq \dots \preceq q_\ell \preceq p_m \preceq q_{\ell-n^2+1} \preceq \dots \preceq q_\ell$.

The instance of the decision version of the SGBLR problem would constitute $\mathcal{D}$, ranking π , and the non-negative integer $m - r$. We defer the soundness and completeness proof to Appendix E.

Remark 11. It is worth noting that SGBLR is a special case of the more general g -GBLR problem, where each group could be of arbitrary size. Thus NP-hardness of SGBLR immediately establishes a similar hardness for g -GBLR. Furthermore, one may note that our reduction works even when there are only two (non-singleton) groups with bonuses.

5 Scalable Frameworks via MILP

The geometric algorithm introduced earlier provides an exact, constructive way of identifying valid rankings through hyperplane arrangements. However, such geometric reasoning scales poorly with the number of data points n and dimensions d , since the number of combinatorial regions grows exponentially. Even for moderate d , enumerating or traversing these regions becomes practically intractable. To design a more scalable alternative, we reformulate g -GBLR problem as a **Mixed Integer Linear Program (MILP)** – a mature framework that allows us to express logical ordering and grouping constraints in a linear-algebraic form, solvable by powerful off-the-shelf engines.

The MILP formulation represents ranking relationships through a set of linear inequalities defined over two classes of variables – continuous feature weights w and binary membership indicators δ_{ir} . Each inequality enforces the required order between tuples in terms of their weighted feature scores and group bonuses. This formulation connects combinatorial ranking constraints with continuous optimization, allowing discrete ordering relations to be reasoned about using standard MILP solvers.

5.1 MILP_{Base} Formulation

We first define the base formulation, denoted as MILP_{Base}, which represents the canonical encoding of the ranking constraints:

$$\begin{aligned}
& \text{minimize} && 0 \\
& \text{subject to} && \sum_{j \in [d]} (t_i[j] - t_{(i+1)}[j]) \cdot w_j + \sum_{r \in [g]} \delta_{ir} \cdot v_r \\
& && - \sum_{r \in [g]} \delta_{(i+1)r} \cdot v_r \geq 0 && \forall i < n \\
& && \sum_{r \in [g]} \delta_{ir} \leq 1 && \forall i \in [n] \\
& && \sum_{i \in [n]} \sum_{r \in [g]} \delta_{ir} \leq k && \delta_{ir} \leq k \\
& && \delta_{ir} \in \{0, 1\}, v_r \geq 0 && \forall i \in [n], r \in [g] \\
& && |w_j| \geq 1 && \forall j \in [d]
\end{aligned}$$

Here, $w \in \mathbb{R}^d$ denotes the feature weights to be learned, v_r denotes the additive group bonus for group G_r . We introduce $g \cdot n$ binary indicator variables δ_{ir} for $i \in [n]$ and $r \in [g]$, where δ_{ir} is 1 if tuple t_i belongs to group G_r , otherwise 0. The constraints encode ordering, group capacity, uniqueness, non-negativity, and weight-magnitude requirements. The objective is to minimize 0 (i.e., this is a feasibility problem), which means the ILP seeks any feasible solution satisfying all constraints that respects the ranking π .

Notice that the MILP_{Base} formulation remains generic and domain-agnostic: it treats all features and groups symmetrically, without leveraging problem-specific insights. In many real-world settings, additional knowledge—such as bounds on group influence, or on pre-identified boosted tuples—can meaningfully restrict the feasible space and guide the solver toward more computationally efficient solutions. Incorporating such application-driven structure motivates a refined formulation, MILP_{Refined}, described next.

5.2 MILP_{Refined} Formulation

MILP_{Refined} – The refined formulation tightens the feasible region by incorporating domain/application-informed assumptions and introducing solver-oriented constraints that eliminate redundant solutions and enhance numerical stability.

MILP_{Refined} introduces the following enhancements:

- (1) **Strict ordering with tolerance:** To ensure a clear and stable ranking, we introduce a small positive margin ϵ between consecutive tuples. This enforces a strictly decreasing order of adjusted scores rather than allowing equality, thereby avoiding ties and improving numerical stability during optimization. We adopt a strategy similar to that of prior work [11]. In our experiments, we set ϵ as 10^{-5} , which ensured sufficient spacing between tuple scores, thus accommodating floating-point tolerances.
- (2) **Bounded additive bonuses:** Each group's additive bonus is constrained within a predefined range to prevent it from dominating the linear utility function. These bounds capture realistic limitations on how much a group-specific adjustment can influence the overall ranking. We emphasize this bound is not a restriction of MILP_{Refined}, and can be changed based on expectations of the range using domain knowledge.
- (3) **Non-negative feature weights:** In many real-world settings, such as performance evaluation or credit scoring, features contribute positively to the final score. Accordingly, all feature weights are constrained to be non-negative, reflecting domain semantics and eliminating redundant sign symmetries in the optimization process. We emphasize that this assumption is not a limitation of MILP_{Refined}; if a feature is known to contribute

negatively to the final utility, one can appropriately scale the feature values to ensure this.

- (4) **Dominated tuples:** Under the assumption that feature weights are non-negative, we use the standard idea of dominance followed in monotone functions [6]. If it is observed that there exists a pair of tuples where t_j dominates t_i , yet t_i is ranked before t_j , then it is guaranteed that t_i must have received an additive bonus. Therefore, we add the constraint that t_i must belong to one of the groups receiving a bonus by adding a constraint $\sum_{r \in [g]} \delta_{ir} = 1$. Finding the set of points that dominate at least one other point yet are ranked lower can be formulated as a skyline query [6]. To this end, the dataset is augmented with an additional dimension representing the rank of each tuple. Specifically, for a tuple t_i , we append its rank in π as the $(d+1)$ th dimension. On the resulting augmented dataset, we compute the skyline—where larger values are preferred in every dimension—using the Double Divide-and-Conquer (DDC) algorithm, which runs in $O(n \log^{d-1} n + nd)$ time [29]. Any tuple that is dominated by another must necessarily receive a bonus. This procedure can be repeated iteratively until no additional dominated tuples are identified.

The complete MILP_{Refined} model is thus:

$$\begin{aligned}
& \text{minimize} && 0 \\
& \text{subject to} && \sum_{j \in [d]} (t_i[j] - t_{(i+1)}[j]) \cdot w_j + \sum_{r \in [g]} \delta_{ir} \cdot v_r \\
& && - \sum_{r \in [g]} \delta_{(i+1)r} \cdot v_r \geq \epsilon && \forall i < n \\
& && \sum_{r \in [g]} \delta_{ir} \leq 1 && \forall i \in [n] \\
& && \sum_{i \in [n]} \sum_{r \in [g]} \delta_{ir} \leq k && \delta_{ir} \leq k \\
& && 0 \leq v_r \leq 100 && \forall r \in [g] \\
& && w_j \geq 0 && \forall j \in [d]
\end{aligned}$$

Relationship Between MILP_{Base} and MILP_{Refined}. The relationship between the two formulations is hierarchical. The feasible region of MILP_{Refined} is a strict subset of that of MILP_{Base}, ensuring that every refined solution remains sound under the base model. While MILP_{Base} defines a broad feasibility space, MILP_{Refined} incorporates additional structure—such as bounded bonuses and restricted membership subsets—that enhances interpretability, aligns with domain semantics, and yields more realistic solutions. In essence, MILP_{Base} serves as the foundational declarative model, whereas MILP_{Refined} is its solver-optimized and practically scalable counterpart.

6 Experimental Evaluation

We now present an empirical evaluation of our proposed formulations and algorithms. The primary goals of this experimental evaluation are to assess scalability and practical applicability, and to compare the proposed methods with one another and with suitable baselines. We examined both real-world and synthetic datasets to understand how the models behave under diverse data distributions and parameter settings.

6.1 Experimental setup

Experimental environment. All experiments were run on a server with an AMD EPYC 7763 CPU, with 128GB of RAM allocated for each experiment, running Ubuntu 24.04. All code is implemented using Python 3.12.3 with Gurobi version 12.0.2 as the MILP solver. Gurobi and implemented algorithms were allowed to use up to 16 threads. The timeout was considered 1800 seconds.

Datasets. We evaluate our methods on one real-world dataset and a set of synthetically generated datasets.

Real-world data. The real-world dataset, introduced in prior work [7], contains examination scores of 384,977 candidates who took the Joint Entrance Examination (JEE) in India in 2009. After removing entries with missing values, 384,970 candidates remain, each described by three numerical attributes—Mathematics, Physics, and Chemistry scores—and categorical attributes such as gender and reservation category. We use the three numerical scores as equally weighted features. Categorical attributes define groups receiving additive bonuses. In single-group experiments, female candidates receive a bonus of 4, reflecting the median score difference from male candidates. In two-group experiments, candidates from underprivileged categories receive a bonus of 6, and female candidates receive a further bonus of 4, corresponding to their median differences from the general (GE) category. These data-driven adjustments preserve the original score distribution while modeling group-level advantages.

Synthetic data. We also construct multiple synthetic datasets to systematically evaluate scalability. Each dataset $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ consists of n tuples, where each tuple t_i is a d dimensional feature vector. Each feature component $t_i[j]$ is sampled independently from a uniform distribution over $[0, 25)$ and rounded to two decimal places. A linear weight vector $W = (w_1, \dots, w_d)$ is sampled similarly. To introduce structured deviations from a purely linear ranking, additive bonuses are assigned to randomly generated groups of tuples. For each group G_r , a bonus v_r is sampled uniformly from $[5d, 10d)$. For MILP_{Refined}, a random subset $\mathcal{D}' \subseteq \mathcal{D}$ is first selected, and groups are then formed by uniformly sampling tuples from $\mathcal{D}'$. Each tuple's augmented score is computed as $f(t_i) = \sum_{j=1}^d w_j t_i[j] + v_r$ if it belongs to G_r , and as $f(t_i) = \sum_{j=1}^d w_j t_i[j]$ otherwise. Finally, tuples are sorted in decreasing order of their augmented scores to yield the target ranking π . This generation process models ranking scenarios in which a base linear utility function is modified by sparse, group-level adjustments, providing a controlled ground truth for evaluation.

Baselines. We consider three baselines for comparison. We use ordinal regression [51] as the first baseline, where ranks are used as the ordinal predictor values. As a second baseline, we apply logistic regression by creating a dataset of all pairs of tuples. For a pair t_i ranked above t_j , the tuple $t_i - t_j$ is labeled 1, and $t_j - t_i$ labeled 0. This results in a supervised learning task with binary labels, on which standard logistic regression can be applied. Both of these baselines output a linear utility function, and given this function, the number of tuples that need to be given a bonus to realize the input function is computed. We also consider a simple baseline, called LocalSearch, which repeatedly samples a random unit vector as the weight function within the given time limit. For each sampled weight vector, it computes the minimum number of tuples that must receive an additive bonus to realize the target ranking π . The best-performing weight vector – i.e., the one minimizing the number of required bonuses – is returned as the final output.

The code is available at the following link ¹. Our experimental evaluation examines both the computational performance and the solution quality of the proposed formulations and algorithms.

¹<https://github.com/Trustworthy-Ranking-Data-Management-lab/explaining-rankings-hidden-bonus>

6.2 Research Questions

Singleton Setting.

RQ1: How do the algorithms perform in the singleton-groups setting, and how does their computational behavior vary with increasing instance size?

One Group of Size k .

RQ2: How does varying the feature dimension d affect the runtime and scalability of the algorithms in the one-group setting?

RQ3: How does changing the number of tuples n influence the computational performance of the algorithms in this setting?

RQ4: How does the group size parameter k impact the efficiency and scalability of the algorithms when there is a single protected group?

Real-World Dataset.

RQ5: How do the algorithms behave with one group setting for a real-world dataset, and how does this configuration affect computational performance?

RQ6: How do the algorithms behave with a two-group setting for a real-world dataset, and how does this configuration affect computational performance compared with the one-group setting?

Quality of Explanations.

RQ7: Across the synthetic datasets, and the real world dataset, do algorithms recover a bonus-aware linear utility function that aligns with the observed ranking?

6.3 Results: Performance and Scalability

Results: RQ1 – Singleton Groups Setting. We experimented with synthetic two-dimensional data, where the number of tuples given an additive bonus (k) was set to 10% of the total number of tuples ($0.1n$). Figure 1a presents the results. As shown, within a time-out of 1800 seconds, ERMB scales only up to 5,000 tuples, highlighting its limited practical applicability. This behavior aligns with its theoretical complexity of $O(n^3 \log n)$ and motivates the need for practical MILP-based reformulations.

For the special case of two-dimensional data, the MILP-based approach MILP_{Refined} scales well, handling up to 75,000 tuples within 450 seconds. However, even MILP_{Refined} fails to scale beyond two dimensions for singleton groups. Detailed results is shown in the appendix (Figure 5), after $n=5,000$ MILP_{Refined} times out for $d=5$.

For the quality analysis of the produced solutions, we compared the outputs of LocalSearch, ERMB, MILP_{Refined}, ordinal regression, and logistic regression whenever solutions were available. Since LocalSearch operates under a fixed time budget and relies on random sampling, the solution quality depends on the allocated runtime. As shown in Figure 1c, for two-dimensional data, LocalSearch and MILP_{Refined} select an almost identical number of tuples to receive an additive bonus (represented as the best value in the figure) for up to 15,000 tuples. However, as n increases, the quality of LocalSearch begins to degrade, even for $d=2$. Therefore, we exclude LocalSearch from the comparison in the remaining research questions. We observe that ordinal regression and logistic regression also find solutions of low quality, requiring a much larger number of tuples to be allocated an additive bonus to realize the observed function. This

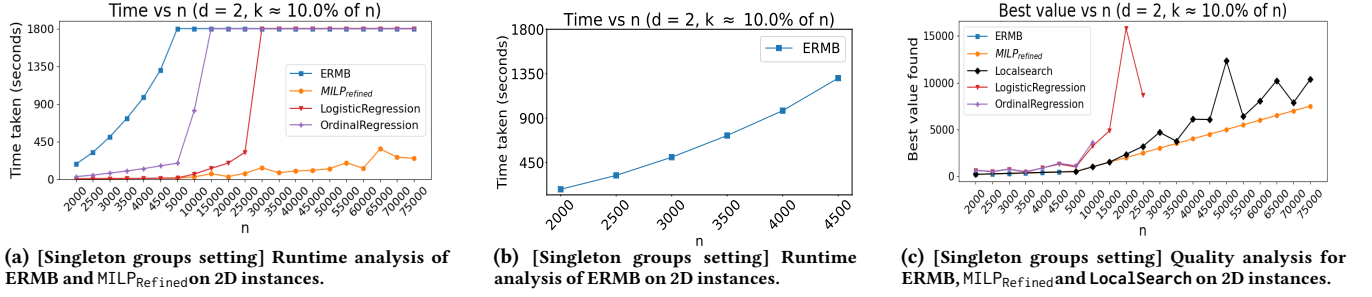


Figure 1: Analysis of ERMB

is not unexpected given that these methods are not designed to account for hidden additive bonuses. Therefore, we also exclude these baselines from the comparison in the remaining research questions.

Results: RQ2, RQ3 and RQ4 – One Group Setting. Considering practical formulation of the MILP-based formulations, MILP_{Base} and MILP_{Refined}, we now turn to RQ2–RQ4, where we study the scalability of these approaches under varying parameters. In particular, we analyze their behavior as we vary the dimensionality d , the number of tuples n , and the group size k .

Regarding RQ2 – As shown in Figures 2a and 2b, for fixed tuple sizes of $n=10,000$ and $n=50,000$ respectively (with k set to $0.1n$), the runtime of both MILP_{Refined} and MILP_{Base} increases as the dimensionality d grows. For $n=10,000$, the impact is particularly evident for MILP_{Base}, which times out beyond $d=5$. In contrast, MILP_{Refined} remains tractable and scales up to $d=17$ within 450 seconds. This demonstrates that incorporating domain-specific structure significantly improves the scalability of MILP_{Refined}.

However, when increasing the dataset size to $n=50,000$, even MILP_{Refined} times out for dimensions above $d=11$. This indicates that while the refined formulation extends scalability, high-dimensional large instances remain computationally challenging at larger scales. An additional scalability challenge in higher dimensions is the degradation of the skyline pruning, which is part of MILP_{Refined}. Skyline pruning is extremely effective at $d=2$, with around 95% of the tuples that are boosted being identified by skyline pruning. When $d=5$, the effect diminishes significantly, with around 5% of the tuples identified, and at $d \geq 8$ the skyline pruning is unable to identify tuples that are definitely boosted.

Regarding RQ3 – For two-dimensional data, the increase in tuple size has only a modest impact on runtime. Although we observe a slight increase in execution time as n grows, both MILP_{Base} and MILP_{Refined} exhibit comparable performance in this setting. The detailed figure is presented in the appendix (Figure 7).

However, for $d=8$, as shown in 2c, the situation changes markedly. Even at $n=2,000$, MILP_{Base} already times out, clearly indicating that dimensionality has a far more significant effect on computational cost than the number of tuples. For $d=8$, MILP_{Refined} remains scalable up to 50,000 tuples; beyond that point, solver-level heuristic changes introduce noticeable performance variance.

Regarding RQ4 – MILP solvers exhibit irregular and non-monotonic behavior: small changes in k often lead to substantial fluctuations in runtime. This behavior is typical for MILPs. Although modifying k only alters the tightness of the capacity constraints $\sum_i \delta_{ir} \leq k$,

such changes can significantly affect the strength of the MILP relaxation, the size of the feasible region for the δ -variables, the degree of symmetry in the model, and ultimately the structure of the branch-and-bound tree explored by the solver. As a result, the runtime may increase or decrease unpredictably. This phenomenon is also observed in our experiments. For example, with $n=10,000$ and $d=5$, MILP_{Base} exhibits pronounced runtime peaks at intermediate values of k (in the range of 10–50%), whereas MILP_{Refined} remains consistently faster while still reflecting the same underlying variability. Similar patterns are observed for $d=8$ and larger values of n . Overall, these results highlight the inherent sensitivity of MILP solvers to constraint tightness. Detailed results are presented in the appendix – Figures 8, 9, and 6.

Results: RQ5, RQ6 – Real World Dataset. We now turn to the real-world dataset containing more than 300,000 tuples.

Regarding RQ5 – As shown in Figure 3, both MILP_{Base} and MILP_{Refined} exhibit similar performance in the single-group setting. As the tuple size increases, the runtime grows approximately linearly with n . Notably, we are able to find a realizable function explaining the ranking for all 300,000 tuples within the 1800-second timeout.

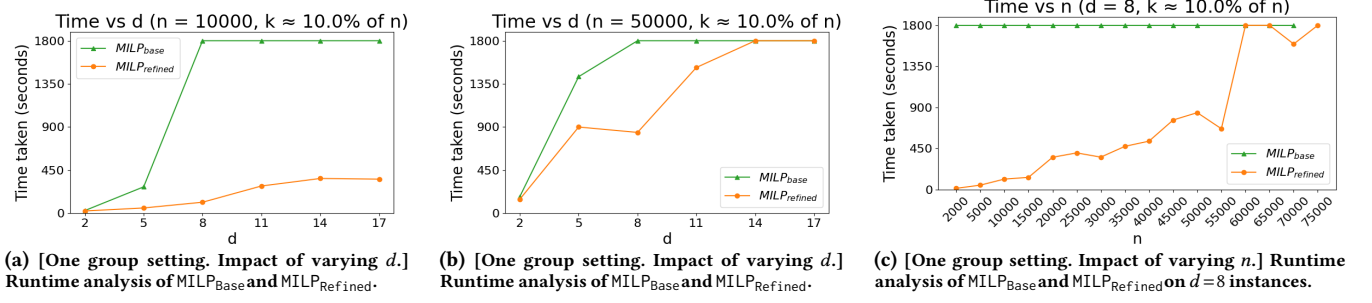
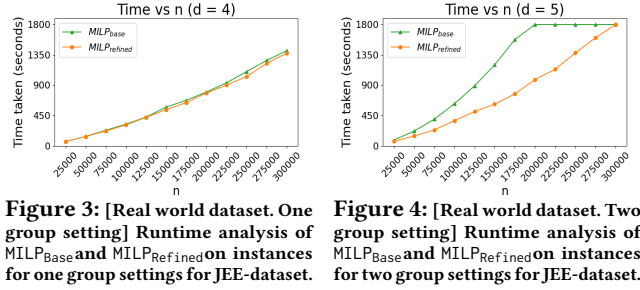
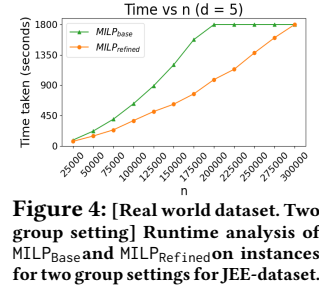
Regarding RQ6 – As shown in Figure 4, in the two-group setting, the domain-knowledge constraints again provide a clear advantage to MILP_{Refined}. While MILP_{Base} scales up to 150,000 tuples, MILP_{Refined} remains tractable up to 275,000 tuples.

Results: RQ7 – Quality of Explanations. Finally, we evaluate explanation quality across all datasets. As mentioned while discussing datasets, for the synthetic datasets, both the data and the underlying linear function are known, allowing a precise assessment of reconstruction accuracy. Similarly, for the JEE dataset, the injected additive bonuses and affected tuples are also known, enabling direct evaluation of recovery accuracy.

Across all synthetic datasets as well as the JEE dataset, the solutions produced by MILP_{Refined} and MILP_{Base} exactly recover the added bonuses and the corresponding tuples. These results indicate that the MILP-based formulations scale effectively to large real-world datasets while maintaining high solution quality.

7 Related Work

Utility functions: Many classes of utility functions have been explored in prior work. *Monotone utility functions* [19] form one such class: given two tuples t_i and t_j where $t_i[x] > t_j[x]$ for every attribute x , the function assigns a higher score to t_i than to t_j . Efficient Top- k evaluation for monotone functions was investigated by Fagin et al. [20], who proposed the Threshold and Fagin’s algorithms. Along

Figure 2: Analysis of impact of varying d (a & b) and varying n (c).Figure 3: [Real world dataset. One group setting] Runtime analysis of MILP_{Base} and MILP_{Refined} on instances for one group settings for JEE-dataset.Figure 4: [Real world dataset. Two group setting] Runtime analysis of MILP_{Base} and MILP_{Refined} on instances for two group settings for JEE-dataset.

similar lines, Xin et al. [60] study Top- k computation under user preferences over selected attributes, modeled as convex functions.

Beyond convexity, *non-linear utility functions* have also been examined, including quadratic functions [27] and more general polynomial functions [41] in multi-criteria Top- k settings. Among the many families of utility functions, *linear functions* have been especially influential, forming a core component of multi-criteria decision making [39], ranking and interactive scoring systems [58, 59], and the interpretation of complex machine learning models [45].

While non-linear, monotone, and other function classes have been used for Top- k generation, work on utility and scoring frameworks [3, 48] has predominantly focused on linear functions. Following this well-established line of research, where linear functions serve as standard models for utility and scoring [39, 48], we also use *linear functions* as the scoring model in our work with additive bonuses.

Learning ranking functions: Chen et al. [11] propose RankHow, which explains an observed ranking using a linear function chosen to minimize Spearman’s footrule distance from π . Their formulation, however, does not allow additive bonuses, and thus may fail to realize π when tuples must be organized into at most g groups receiving group-wise boosts, as required in g -GBLR. Fair ranking approaches [3, 7] also learn linear utility functions but focus solely on satisfying group fairness constraints in the top- k positions, without modeling additive adjustments.

Work on reverse top- k queries [56] is also related, seeking linear functions under which specified tuples appear in the top- k . Why-not queries [25] similarly search for a linear function that places a designated tuple within the top- k , while other formulations [32] assume a fixed set of linear functions and aim to minimally modify tuples so that they enter or exit the top- k . These approaches, however, do not incorporate additive bonuses and thus do not generalize to our setting; moreover, they only explain top- k behavior for selected tuples and do not address the problem of explaining a complete ranking.

Traditional regression techniques – such as linear models [35], decision trees [9], and neural networks [30] – require numeric labels and thus cannot be applied when only an ordering is available. Additionally, extension of these techniques to make them bonus-aware is not straightforward and can be an interesting work in itself. Model-agnostic explanation tools like LIME [45] and its related IR approaches [55], produce explanations by querying the underlying model through sampling of data points around the query point, which is infeasible in our setting where the ranking function is unknown.

Our work is the first to study the problem of bonus-aware linear ranking functions as a model of explanation of a given ranking.

8 Conclusion and Future Work

In this work, we look at explanations to ranking problems based on linear scoring functions with an additive bonus model. An interesting direction is to extend or design new methods for other classes of ranking functions, such as quadratic/non-linear utility functions. Another promising open direction is to consider different models for bonus, for instance, a multiplicative bonus model, where each tuple’s bonus would be computed by $f_w(t) = \text{bonus}(t[\tau]) \cdot \left(\sum_{j=1}^d w_j \cdot t[j] \right)$.

While our ERMB approach and the hardness results apply also to the multiplicative model, our extension to g -GBLR, and the ILP-based approaches do not extend. An interesting avenue of research is to design efficient algorithms for the multiplicative-bonus setting.

In this paper, we show that no efficient algorithm can exactly solve the bonus-aware linear ranking explanation problem. This hardness result suggests a natural next step: can we design algorithms that achieve a small approximation (in the number of bonus groups)? Or, under standard complexity assumptions, can we prove that even such approximations are impossible? The latter appears especially relevant given our MILP formulation, which points to a close connection with the classical Max- k -LIN problem, for which strong hardness-of-approximation results are known (e.g., [2, 5, 28, 40]). This, in turn, raises the question of whether approximation algorithms for the bonus-aware linear ranking explanation problem are likewise unattainable.

Acknowledgments

The work of Suraj Shetiya was supported by ANRF under Grant ANRF/ECRG/2024/004976/ENS. The work of Sujoy Bhore was supported by ANRF ARG-MATRICES, Grant 002465. The work of Dip-tarka Chakraborty was supported in part by an MoE AcRF Tier 1 grant (T1 251RES2303) and a Google South & South-East Asia Research Award. The work of Priyanka Golia was supported by ANRF early career grant ANRF/ECRG/2024/005777/ENS.

References

- [1] Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W Bruce Croft. 2018. Unbiased learning to rank with unbiased propensity estimation. In *SIGIR 2018*.
- [2] Sanjeev Arora, László Babai, Jacques Stern, and Z Sweedyk. 1997. The hardness of approximate optima in lattices, codes, and systems of linear equations. *J. Comput. System Sci.* 54, 2 (1997), 317–331.
- [3] Abolfazl Asudeh, Hosagrahar Visvesvaraya Jagadish, Julia Stoyanovich, and Gautam Das. 2019. Designing fair ranking schemes. In *SIGMOD 2019*. 1259–1276.
- [4] Abolfazl Asudeh, Azade Nazi, Nan Zhang, Gautam Das, and HV Jagadish. 2019. RRR: Rank-regret representative. In *SIGMOD 2019*. 263–280.
- [5] Amey Bhargale and Subhash Khot. 2021. Optimal inapproximability of satisfiable k-LIN over non-abelian groups. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. 1615–1628.
- [6] Stephan Börzsönyi, Donald Kossmann, and Konrad Stocker. 2001. The skyline operator. In *ICDE 2001*. IEEE.
- [7] Guangya Cai. 2025. Finding a Fair Scoring Function for Top-k Selection: From Hardness to Practice. arXiv:2503.11575 [cs.DB] <https://arxiv.org/abs/2503.11575>
- [8] Yuan-Chi Chang, Lawrence Bergman, Vittorio Castelli, Chung-Sheng Li, Ming-Ling Lo, and John R Smith. 2000. The onion technique: Indexing for linear optimization queries. In *ACM SIGMOD 2000*. 391–402.
- [9] Bahzad Charbuty and Adnan Abdulazeez. 2021. Classification based on decision tree algorithm for machine learning. *Journal of applied science and technology trends* 2, 01 (2021), 20–28.
- [10] Qixu Chen and Raymond Chi-Wing Wong. 2024. Robust Best Point Selection under Unreliable User Feedback. *Proceedings of the VLDB Endowment* 17, 11 (2024).
- [11] Zixuan Chen, Panagiotis Manolios, and Mirek Riedewald. 2025. Synthesizing Scoring Functions for Rankings using Symbolic Gradient Descent. In *ICDE 2025*.
- [12] Tanya Chowdhury, Razieh Rahimi, and James Allan. 2023. Rank-lime: local model-agnostic feature attribution for learning to rank. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*. 33–37.
- [13] Tanya Chowdhury, Yair Zick, and James Allan. 2025. RankSHAP: Shapley value based feature attributions for learning to rank. In *International Conference on Learning Representations*, Vol. 2025. 36765–36794.
- [14] Gautam Das, Dimitrios Gunopulos, Nick Koudas, and Dimitris Tsirgiannis. 2006. Answering top-k queries using views. In *PVLDB 2006*. 451–462.
- [15] Andrew Davenport and Jayant Kalagnanam. 2004. A computational study of the Kemeny rule for preference aggregation. In *AAAI*, Vol. 4. 697–702.
- [16] Finale Doshi-Velez and Been Kim. 2017. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608* (2017).
- [17] Cynthia Dwork, Ravi Kumar, Moni Naor, and Dandapani Sivakumar. 2001. Rank aggregation methods for the web. In *WWW 2001*. 613–622.
- [18] Herbert Edelsbrunner. 1987. *Algorithms in combinatorial geometry*. Vol. 10. Springer Science & Business Media.
- [19] Ronald Fagin. 2002. Combining fuzzy information: an overview. *ACM SIGMOD Record* 31, 2 (2002), 109–118.
- [20] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *Proceedings of the twentieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 102–113.
- [21] Abraham Gale and Amélie Marian. 2020. Explaining monotonic ranking functions. *Proc. VLDB Endow.* 14, 4 (Dec. 2020), 640–652. doi:10.14778/3436905.3436922
- [22] Abraham Gale and Amélie Marian. 2024. Explainable disparity compensation for efficient fair ranking. In *ICDE 2024*. IEEE.
- [23] Hongjie Guo, Jianzhong Li, and Hong Gao. 2025. Towards strong regret minimization sets: Balancing freshness and diversity in data selection. *Theoretical Computer Science* 1026 (2025), 114986.
- [24] Qianyu Guo, Gong Haoteng, Xujun Wei, Yanwei Fu, Yizhou Yu, Wenqiang Zhang, and Weifeng Ge. 2023. Rankdnn: Learning to rank for few-shot learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 728–736.
- [25] Zhian He and Eric Lo. 2014. Answering Why-Not Questions on Top-K Queries. *IEEE Transactions on Knowledge and Data Engineering* 26, 6 (2014), 1300–1315.
- [26] Maria Heuss, Maarten de Rijke, and Avishek Anand. 2025. RankingSHAP-Faithful Listwise Feature Attribution Explanations for Ranking Models. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 381–391.
- [27] Taylor Kessler Faulkner, Will Brackenbury, and Ashwin Lall. 2015. K-regret queries with nonlinear utilities. *Proceedings of the VLDB Endowment* 8, 13 (2015).
- [28] Subhash Khot, Guy Kindler, Elchanan Mossel, and Ryan O'Donnell. 2007. Optimal inapproximability results for MAX-CUT and other 2-variable CSPs? *SIAM J. Comput.* 37, 1 (2007), 319–357.
- [29] Hsiang-Tsung Kung, Fabrizio Luccio, and Franco P Preparata. 1975. On finding the maxima of a set of vectors. *Journal of the ACM (JACM)* 22, 4 (1975), 469–476.
- [30] Chiyan Lee, Hideyuki Hasegawa, and Shangee Gao. 2022. Complex-Valued Neural Networks: A Comprehensive Survey. *IEEE/CAA Journal of Automatica Sinica* 9 (2022), 1406–1426. <https://api.semanticscholar.org/CorpusID:251324986>
- [31] Hao Liu, Raymond Chi-Wing Wong, Zheng Zhang, Min Xie, and Bo Tang. 2024. Fair Top-k Query on Alpha-Fairness. In *ICDE 2024*. IEEE, 2338–2350.
- [32] Qing Liu, Yunjun Gao, Gang Chen, Baihua Zheng, and Linlin Zhou. 2016. Answering why-not and why questions on reverse top-k queries. *The VLDB Journal* 25, 6 (2016), 867–892.
- [33] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 4768–4777.
- [34] Michael Mathioudakis, Carlos Castillo, Giorgio Barnabo, and Sergio Celis. 2020. Affirmative action policies for top-k candidates selection: with an application to the design of policies for university admissions. In *ACM SAC 2020*.
- [35] Dastan Maulud and Adnan M Abdulazeez. 2020. A review on linear regression comprehensive in machine learning. *Journal of applied science and technology trends* 1, 2 (2020), 140–147.
- [36] Nimrod Megiddo. 1984. Linear programming in linear time when the dimension is fixed. *Journal of the ACM (JACM)* 31, 1 (1984), 114–127.
- [37] Marco Morik, Ashudeep Singh, Jessica Hong, and Thorsten Joachims. 2020. Controlling fairness and bias in dynamic learning-to-rank. In *SIGIR 2020*. 429–438.
- [38] Danupon Nanongkai, Ashwin Lall, Atish Das Sarma, and Kazuhisa Makino. 2012. Interactive regret minimization. In *SIGMOD 2012*. 109–120.
- [39] Danupon Nanongkai, Atish Das Sarma, Ashwin Lall, Richard J Lipton, and Jun Xu. 2010. Regret-minimizing representative databases. *PVLDB 2010* (2010).
- [40] Ryan O'Donnell, Yi Wu, and Yuan Zhou. 2011. Hardness of Max-2Lin and Max-3Lin over integers, reals, and large cyclic groups. In *IEEE CCC 2011*. IEEE.
- [41] Jianzhong Qi, Fei Zuo, Hanan Samet, and Jia Cheng Yao. 2018. K-regret queries using multiplicative utility functions. *ACM TODS 2018* (2018).
- [42] Li Qian, Jinyang Gao, and HV Jagadish. 2015. Learning user preferences by adaptive pairwise comparison. *Proceedings of the VLDB Endowment* 8, 11 (2015), 1322–1333.
- [43] Miroslav Rada and Michal Cerny. 2018. A new algorithm for enumeration of cells of hyperplane arrangements and a comparison with Avis and Fukuda's reverse search. *SIAM Journal on Discrete Mathematics* 32, 1 (2018), 455–473.
- [44] Theresia Veronika Rampisela, Tuukka Ruotsalo, Maria Maistro, and Christina Lioma. 2024. Can we trust recommender system fairness evaluation? the role of fairness and relevance. In *SIGIR 2024*. 271–281.
- [45] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should I trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*.
- [46] Rolf Schneider. 2022. *Convex cones: geometry and probability*. Vol. 2319. Springer.
- [47] Procheta Sen, Debasis Ganguly, Manisha Verma, and Gareth JF Jones. 2020. The curious case of IR explainability: Explaining document scores within and across ranking models. In *SIGIR 2020*. 2069–2072.
- [48] Suraj Shetiya, Abolfazl Asudeh, Sadia Ahmed, and Gautam Das. 2019. A unified optimization algorithm for solving "regret-minimizing representative" problems. *Proceedings of the VLDB Endowment* 13, 3 (2019).
- [49] Suraj Shetiya, Ian P Swift, Abolfazl Asudeh, and Gautam Das. 2022. Fairness-aware range queries for selecting unbiased data. In *ICDE 2022*. IEEE.
- [50] Ashudeep Singh and Thorsten Joachims. 2019. Policy learning for fairness in ranking. *Advances in neural information processing systems* 32 (2019).
- [51] V Srinivasan. 1976. Linear programming computational procedures for ordinal regression. *Journal of the ACM (JACM)* 23, 3 (1976), 475–487.
- [52] Sabine Storandt and Stefan Funke. 2019. Algorithms for average regret minimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33.
- [53] Thibaut Thonet, Yagmur Gizem Cinar, Eric Gaussier, Minghan Li, and Jean-Michel Renders. 2022. Listwise learning to rank based on approximate rank indicators. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 8494–8502.
- [54] Csaba D Toth, Joseph O'Rourke, and Jacob E Goodman. 2017. *Handbook of discrete and computational geometry*. CRC press.
- [55] Manisha Verma and Debasis Ganguly. 2019. LIRME: locally interpretable ranking model explanation. In *SIGIR 2019*. 1281–1284.
- [56] Akrivi Vlachou, Christos Doukeridis, Yannis Kotidis, and Kjetil Nørsvåg. 2010. Reverse top-k queries. In *ICDE 2010*. IEEE, 365–376.
- [57] Weicheng Wang, Raymond Chi-Wing Wong, HV Jagadish, and Min Xie. 2024. Reverse regret query. In *ICDE 2024*. IEEE.
- [58] Weicheng Wang, Raymond Chi-Wing Wong, Jinyang Li, and HV Jagadish. 2025. Interactive Learning for Diverse Top-k Set. In *ICDE 2025*. IEEE Computer Society.
- [59] Weicheng Wang, Raymond Chi-Wing Wong, and Min Xie. 2021. Interactive search for one of the top-k. In *SIGMOD 2021*.
- [60] Dong Xin, Jiawei Han, Hong Cheng, and Xiaolei Li. 2006. Answering top-k queries with multi-dimensional selections: The ranking cube approach. In *VLDB. VLDB Endowment*, Seattle, WA, USA, 463–475.
- [61] Guangyi Zhang, Nikolaj Tatti, and Aristides Gionis. 2023. Finding favourite tuples on data streams with provably few comparisons. In *ACM SIGKDD 2023*.

Appendix

A Additional experimental details

We additionally performed experiments where the tuple feature components and weight vector components are sampled from a Zeta

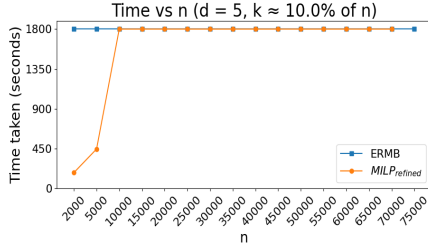


Figure 5: [Singleton group setting] Runtime analysis of ERMB and $MILP_{\text{Refined}}$ on $d = 5$ instances for singleton group settings. k is fixed to 10% of tuples n .

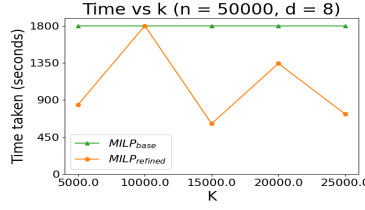


Figure 6: [One group setting. Analysis of the impact of varying k .] Runtime analysis of $MILP_{\text{Base}}$ and $MILP_{\text{Refined}}$ on $d = 8$ and $n = 50000$ instances for one group settings.

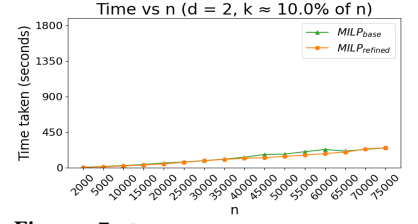


Figure 7: [One group setting. Analysis of the impact of varying n .] Runtime analysis of $MILP_{\text{Base}}$ and $MILP_{\text{Refined}}$ on 2D instances.

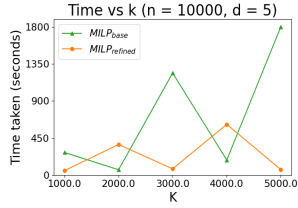


Figure 8: [One group setting. Analysis of the impact of varying k .] Runtime analysis of $MILP_{\text{Base}}$ and $MILP_{\text{Refined}}$ on $d = 5$ and $n = 10000$ instances for one group settings.

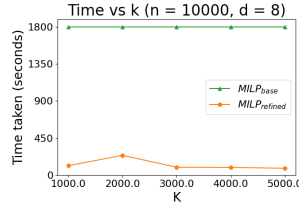


Figure 9: [One group setting. Analysis of the impact of varying k .] Runtime analysis of $MILP_{\text{Base}}$ and $MILP_{\text{Refined}}$ on $d = 8$ and $n = 10000$ instances for one group settings.

(zipfian) distribution with parameter 2. Figures 10a, 10b and 10c show the results. The running time of both $MILP_{\text{Base}}$ and $MILP_{\text{Refined}}$ are significantly faster on these instances as compared to when data was drawn from the uniform distribution (as shown in figures 2a, 2b, 2c). The nature of the zipfian distribution increases the efficacy of the skyline pruning operation significantly, where at $d = 8$ approximately 80% of the boosted tuples can be identified.

B Discussion on Positive Weights

In practice, there are more restrictions on the weights in the function. In many applications, each attribute either positively or negatively contributes to the overall score. In our running example, a user would be interested in obtaining the *best* quality computer at the *least* price. To uniformly deal with such cases, we normalize the attribute score of “price” (in general, any negatively contributing attribute A), of each item t by

$$t[A] = \frac{\max_j(t_j[A]) - t[A]}{\max_j(t_j[A]) - \min_j(t_j[A])}$$

Once normalized, the weight of each attribute in the utility function can be restricted to positive values. While our techniques in the paper can handle both positive and negative weights, careful observations and optimizations based on this restriction are presented throughout the paper.

C Missing Details of ERMB

LEMMA 12. *ERMB (Algorithm 1) has a time complexity of $O(n^{2d+1} \log n)$.*

PROOF. Algorithm 1 introduces a hyperplane for every pair of tuples t_i, t_j . As stated before, this forms a central arrangement and partitions the d -dimensional space into $n^{2(d-1)}$ disjoint regions. In our $(d-1)$ dimensional space, enumeration of each of the cells can

be performed using the enumeration algorithm in Rada et al. [43]. [43] computes a Linear Program with d variables and n^2 constraints in every cell. Meggido [36] proposes a linear-time algorithm for solving an LP in fixed dimension d . Combining the two results, the enumeration process consumes a total of $O(n^{2d})$ time.

In every region, using the weight vector returned by the LP, the scores for the n tuples are computed in $O(nd)$ time. Based on the scores, sorting is performed to obtain a ranking ρ . Thus, in every region, $O(n \log n)$ time is consumed to construct the ranking ρ . Upon computing the ranking ρ , LIS is computed on ρ in $O(n \log n)$ time. Thus, the overall time spent in a single region is $O(n \log n)$ time. As there are n^{2d-2} regions, our algorithm consumes a total of $O(n^{2d+1} \log n)$ time. $\square$

D Details of the Algorithms for g -GBLR

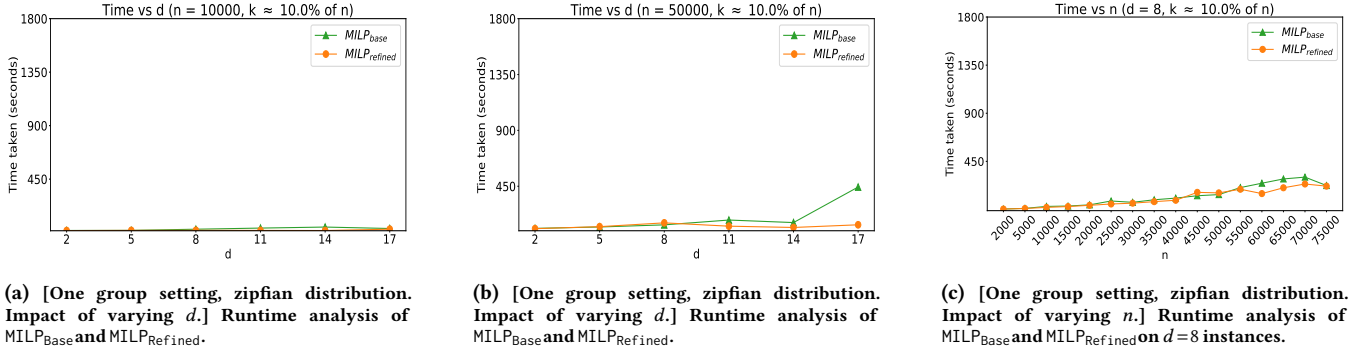
THEOREM 13. *There exists a deterministic algorithm that, given a set of tuples $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ where each $t_i \in \mathbb{R}^d$, a ranking π over $\mathcal{D}$ and an integer k , if there exists a satisfying solution to g -GBLR, returns a vector $w \in \mathbb{R}^d$, $v_1, \dots, v_g \in \mathbb{R}^+$ and disjoint $G_1, \dots, G_g \subseteq \mathcal{D}$ with $\sum_{i \in [g]} |G_i| \leq k$ which satisfies the problem, in time $O(g^{2(d+g)} \cdot n^{2(d+g)})$.*

Embedding group bonus as new dimension: Consider a dataset $\mathcal{D}$, ranking π and a budget g on the number of groups. Additionally, there is a budget on the sum of the sizes of the groups, $\sum_{i \in [g]} |G_i| \leq k$.

To illustrate our idea, consider all tuples t_j that belong to the group G_i with a bonus value of b_i . If the unknown weight vector using the linear utility function was $w' = \{w'_1, w'_2, \dots, w'_d\}$, then every tuple $t_j \in G_i$ would be scored as follows: $\forall t_j \in G_i, f_{w'}(t_j) = b_i + \sum_{a=1}^d w'_a \cdot t_j[a]$.

An observation about the above equation is that b_i is an additive bonus and thus for all tuples t_j that belong to G_i , we can imagine this as a “bonus dimension”. Therefore, for every group G_i , we introduce a new dimension $d+i$. All tuples $t_j \in G_i$ would have the bonus dimension set to 1 while other tuples would have it set to 0. Hence, there are a total of $d+g$ dimensions in our modified setting.

But, two main things need to be handled (i) the tuples that belong to the different groups are not known beforehand, and (ii) the tuples $t_i \in \mathcal{D}$ are still in d -dimensional space. One observation is that we need a uniform treatment of all the tuples in the dataset. Thus, for every tuple t_j , we create $g+1$ copies of the tuple. The first copy corresponds to a version of the tuple t_j which receives no bonus. For this version of the tuple, $t_j^{(0)}$, all bonus dimensions are set to 0 while the original d dimensions remain the same as before. For the g other versions, as t_j can only belong to at most one group, the version i of the tuple, $t_j^{(i)}$ has the $d+i$ dimension set to 1 and the remaining bonus dimensions set to 0. For all versions of the tuple t_i in $d+g$

Figure 10: Analysis of impact of varying d (a & b) and varying n (c).**Algorithm 2** MGRA: Multi-group Ranking Algorithm

```

1: procedure MGRA(Dataset  $\mathcal{D}$ , ranking  $\pi$ , no. of groups  $g$ , size bound  $k$ )
2: Initialize  $\mathcal{D}_G \leftarrow \emptyset$ 
3: for all tuples  $t_j \in \mathcal{D}$  do
4:   Create  $t_j^{(0)}$  with  $d$  dims and  $g$  bonus dims set to 0
5:   Add  $t_j^{(0)}$  to  $\mathcal{D}_G$ 
6:   for  $i=1$  to  $g$  do
7:     Create  $t_j^{(i)}$  with attribute  $d+i \leftarrow 1$ , others 0
8:     Add  $t_j^{(i)}$  to  $\mathcal{D}_G$ 
9: Compute hyperplanes  $\mathcal{H}_G$  in  $\mathbb{R}^{d+g}$  from all pairs in  $\mathcal{D}_G$ 
10: Enumerate all ranking regions  $\mathcal{R}_G$  induced by  $\mathcal{H}_G$ 
11: Initialize  $\rho^* \leftarrow \text{null}$ ,  $\text{maxLCS} \leftarrow 0$ ,  $\mathcal{G} \leftarrow \text{null}$ 
12: for all  $\rho \in \mathcal{R}_G$  do
13:   Compute  $L$ , constrained LCS between  $\rho$  and  $\pi$  with size bound  $k$ 
     using MULTIPLEGROUPLCS
14:   if  $|L| > \text{maxLCS}$  then
15:      $\text{maxLCS} \leftarrow |L|$ ;  $\rho^* \leftarrow \rho$ ;  $\mathcal{G} \leftarrow G$  from LCS output
16: return linear function  $s^{-1}(\rho^*)$ ,  $\mathcal{G}$ 

```

Algorithm 3 MULTIPLEGROUPLCS

```

1: procedure MULTIPLEGROUPLCS( $\pi, \sigma$ )
2: Initialize table  $\text{DP}[n][ (g+1) \cdot n ][k]$  to -1.
3:  $s \leftarrow \text{Solve}(n, (g+1) \cdot n, k)$ 
4: return  $s$ 
5: procedure SOLVE( $i, j, b$ )
6: if  $i=0$  or  $j=0$  then
7:   return 0
8: if  $\text{DP}[i][j][b] > -1$  then
9:   return  $\text{DP}[i][j][b]$ 
10: if  $\pi(i) = p$  and  $\sigma(j) = p^{(t)}$  and  $b > 0$  then
11:    $\text{DP}[i][j][b-1] = \max(\text{Solve}(i-1, j-1, b-1) + 1, \text{Solve}(i, j-1, b))$ 
12: else if  $\pi(i) = p$  and  $\sigma(j) = p^0$  then
13:    $\text{DP}[i][j][b] = 1 + \text{Solve}(i-1, j-1, b)$ 
14: else
15:    $\text{DP}[i][j][b] = \text{Solve}(i, j-1, b)$ 
16: return  $\text{DP}[i][j][b]$ 

```

dimensional space, the first d dimensions are the same as those in $\mathcal{D}$. We refer to this new dataset with $(g+1)n$ tuples as $\mathcal{D}_G$.

Multi-group version of our algorithm: Our idea is to adapt ERMB (Algorithm 1) to this setting using $\mathcal{D}_G$. For every two pairs of tuples $t_i^{(x)}, t_j^{(y)}$ in $\mathcal{D}_G$, we introduce a comparison hyperplane in

$d+g$ dimensional space. These $\binom{n(g+1)}{2}$ hyperplanes denoted as $\mathcal{H}_G$ produce a central arrangement which partitions the space into $(n(g+1))^{2(d+g-1)}$ regions. Similar to ERMB, any weight function in a region produces the same ranking of the $n(g+1)$ tuples. Thus, we enumerate each region and generate the set of all possible rankings $\mathcal{R}_G$.

As there are $g+1$ copies of a tuple, unlike ERMB, this instance of the sub-problem does not correspond to an instance of the Longest Increasing Subsequence (LIS) problem, needing us to focus on the more general Longest Common Subsequence (LCS) problem. Indeed, given two ordered lists π and ρ , an LCS between π and ρ gives us the information about the groups. Thus, as a final step, we perform a *budgeted* variant of LCS between $\rho \in \mathcal{R}$ and π , such that entries in π can match any copy of the same tuple in ρ , while respecting the budget constraints. The constrained version of LCS is solved by a DP with $n \times n(g+1)$ variables while maintaining a state of n^g . We present pseudo-code of MGRA (Algorithm 2) for the g -GBLR problem and in Algorithm 3 for the budgeted variant of the LCS problem.

Note that, similar to ERMB, our MGRA approach is amenable to transforming the equations from a d dimensional space to a $(d-1)$ dimensional space. Our analysis makes use of this transformation in producing the runtime complexity.

D.0.1 Analysis of MGRA. We now present arguments for correctness and time complexity bounds for our algorithm.

THEOREM 14. *There exists an algorithm that finds a satisfying solution to g -GBLR. The algorithm runs in time $O(g^{2(d+g)+1} \cdot n^{2d+3g+2})$.*

Running time.

LEMMA 15. *The running time of MGRA (Algorithm 2) is $O(g^{2(d+g)} \cdot n^{2(d+g)})$.*

PROOF. Observe that for each point in $\mathcal{D}$, we create g additional points in $\mathcal{D}_G$. So $|\mathcal{D}_G| = n(g+1)$. As the dimension of the weight-space has changed to $d+g$ with $(n(g+1))^2$ hyperplanes, the number of possible regions of the central arrangement is $O((n(g+1))^{2(d+g)})$. Similar to Theorem 12, the regions are enumerated using [43], with the LP consuming $O(n^2 g^2)$ time per region, and then solving the LCS problem on the realized ranking ρ .

Now consider the running time of the DP to compute the LCS. It holds that $k \leq n$. Therefore, the number of states in the DP is upper bounded by $O(gn^2)$, and each state is solvable in $O(1)$ time.

Therefore, the overall runtime of the algorithm is upper bounded by $O(g^{2(d+g)} \cdot n^{2(d+g)})$. $\square$

LEMMA 16. *Given $\mathcal{D}, \pi, k$, if there exists a satisfying solution to g -GBLR, then MGRA (Algorithm 2) returns a vector $w \in \mathbb{R}^d$, $v_1, \dots, v_g \in \mathbb{R}^+$ and disjoint $G_1, \dots, G_g \subseteq \mathcal{D}$ with $\sum_{i \in [g]} |G_i| \leq k$ which satisfies the problem.*

PROOF. Suppose that there does exist some vector w^* , groups $G_1^*, \dots, G_g^* \subseteq \mathcal{D}$, and additive values $v_1^*, \dots, v_g^*$ that satisfy the given input.

Consider the region in the arrangement of $\mathcal{H}$ where w^* is located. Let v^* be the optimal bonus scores. Consider the ranking σ over $\mathcal{D}_G$ caused by this partition. As w^* and v^* must lie in one of the regions of the arrangement, an LCS of ρ and π must definitely be explored. The constrained LCS problem between ρ and π can be solved exactly. Based on this, we can obtain both the ranking function w^* and the bonus scores v^* along with the group information. Hence, our algorithm finds a satisfying solution if one exists. $\square$

E Missing Details from the Hardness Results

The following lemma establishes that a YES instance of MAX-1-IN-2-SAT maps to that of SGBLR.

LEMMA 17. *If there exists an assignment that satisfies at least r clauses of the formula F by making exactly one literal in each of these r clauses true, then there exist a weight vector $w \in \mathbb{R}^n$ and at most $m-r$ singleton groups such that π is realized by the linear utility function s_w using an additive bonus of either 2 or -2 to each point in those singleton groups.*

PROOF. Let τ be an assignment that satisfies at least r clauses of the formula F by making exactly one literal in each of these r clauses true. Then consider the following $w \in \mathbb{R}^n$: For each $j \in [n]$,

$$w[j] = \begin{cases} 1 & \text{if } \tau \text{ sets the variable } x_j \text{ to true} \\ -1 & \text{otherwise.} \end{cases}$$

Note, that for each clause C_i in F , if τ satisfies it by making exactly one literal true, then $s_w(p_i) = 0$; otherwise, $s_w(p_i)$ would be either -2 or 2 . Further, for each q_s , for $s \in [\ell]$, $s_w(q_s) = 0$.

Next, consider a singleton group for each point p_i such that $s_w(p_i) \neq 0$. Observe that the number of such singleton groups is at most $m-r$. Furthermore, an additive bonus of either 2 or -2 suffices to make their score equal to 0, and thus with those bonuses to at most $m-r$ singleton group elements, π is realized by the utility function s_w . $\square$

Next, we argue that a NO instance of MAX-1-IN-2-SAT maps to that of SGBLR.

LEMMA 18. *If there exist a weight vector $w \in \{\mathbb{R} \setminus \{0\}\}^n$ and at most $m-r$ singleton groups such that π is realized by the linear utility function s_w using an additive bonus to each point in those singleton groups, then there exists an assignment that satisfies at least r clauses of the formula F by making exactly one literal in each of these r clauses true.*

PROOF. Let us consider a weight $w \in \{\mathbb{R} \setminus \{0\}\}^n$ such that with additive bonuses to the minimum number of singleton groups, π is realized by s_w . Let $\mathcal{G}$ be the set of such singleton groups. By our assumption in the lemma, $|\mathcal{G}| \leq m-r$. We would like to claim that any such singleton group (i.e., $\in \mathcal{G}$) must contain p_i , for some $i \in [m]$, but not any q_j 's.

To argue that, for the sake of contradiction, let there exist $G \in \mathcal{G}$ such that $G = \{q_j \mid \text{for some } j \in [\ell]\}$. Let $j = j_1 n^2 + j_2$, for some $j_1 \in [m+1]$, $j_2 \in [n^2]$. Then there must exist n^2 many groups in $\mathcal{G}$ – a group for each point $q_{j_1 n^2 + t}$, for each $t \in [n^2]$. Otherwise, if any of these groups is not present in $\mathcal{G}$, one can exclude $G = \{q_j\}$ from the set of singleton bonus groups $\mathcal{G}$ and still π is realized by s_w (this is because for all q_i , $s_w(q_i) = 0$), thus violating the minimality of size of $\mathcal{G}$. Now, since $|\mathcal{G}| \leq m-r$, it cannot contain any singleton group with point q_j , for some $j \in [\ell]$.

Next, we claim that if for some p_i , $\{p_i\} \notin \mathcal{G}$, we must have that $s_w(p_i) = 0$. Otherwise, by an argument similar to the above, we can show that $\mathcal{G}$ must contain n^2 many singleton groups, thus violating the assumption $|\mathcal{G}| \leq m-r$.

Now, consider any p_i , such that $\{p_i\} \notin \mathcal{G}$. By the construction, p_i has two non-zero coordinates, say $a, b \in [n]$. Then $w_a = -w_b$. Next, consider the corresponding clause C_i . Observe, C_i would be satisfied by making exactly one literal of it true, by assigning $x_a = \text{true}$ and $x_b = \text{false}$, or $x_a = \text{false}$ and $x_b = \text{true}$.

Now, construct an assignment τ as follows: It sets $x_a = \text{true}$ if $w_a > 0$; otherwise $x_a = \text{false}$. Also, consider another assignment $\bar{\tau}$, which is obtained by flipping the assignment of each variable in τ . By the argument in the previous paragraph, observe that either τ or $\bar{\tau}$ satisfies all the clauses C_i for which $\{p_i\} \notin \mathcal{G}$, by making exactly one literal of each of them to true, and this concludes the proof of the lemma. $\square$

PROOF OF THEOREM 8. It is straightforward to see that the reduction from the MAX-1-IN-2-SAT to SGBLR, which we have provided, is polynomial time. The correctness of the reduction immediately follows from Lemma 17 and Lemma 18, and that completes the proof of Theorem 8. $\square$